

2.1 - Principi delle applicazioni di rete

Il cuore dello sviluppo delle applicazioni di rete è costituito dalla creazione dei programmi che sono eseguiti dai sistemi periferici e che comunicano tra loro via rete, che siano:

- Applicazioni web
Due Programmi diversi che comunicano tra di loro
 - il browser eseguito dall'host dell'utente e
 - il web server che si trova nell'host detto "web server"
- Applicazioni di video on Demand ES - NETFLIX
in cui c'è:
 - un programma fornito dal fornitore dell'applicazione che viene eseguito sul dispositivo dell'utente e
 - un programma server in esecuzione sul nostro server del fornitore del servizio

Server che spesso sono ospitati in data center

Per quanto riguarda la creazione dei programmi, dovremmo scrivere un software in grado di funzionare su più macchine, eccetto nei dispositivi del nucleo della rete quali:

- | | | | | |
|---|---|-------------------------------------|---|--|
| • router o |] | considerando che questi non |] | Un confinamento che però ha facilitato lo sviluppo di una vasta gamma di applicazioni per Internet |
| • commutatori a livello di collegamento |] | lavorano a livello di applicazione, |] | |
| |] | ma ai livelli inferiori |] | |

2.1.1 - Architettura delle applicazioni di rete

Nello sviluppo di un'applicazione di rete, prima di occuparsi della codifica software, è necessario:

- avere in mente un progetto dettagliato **dell'architettura dell'applicazione e**
- **stabilire la sua organizzazione sui vari sistemi periferici**

In merito all'ultimo punto, esistono due principali architetture di rete attualmente utilizzate:

- **l'architettura client-server e**
- **quella peer-to-peer (P2P)**

Nell'**architettura client-server** (client-server architecture) vi è:

- un host sempre attivo chiamato server che
- risponde alle richieste di servizio di molti altri host detti client

ES - applicazioni web

Il server, quando riceve una richiesta di un oggetto da parte di un client, risponde inviando

In essa inoltre:

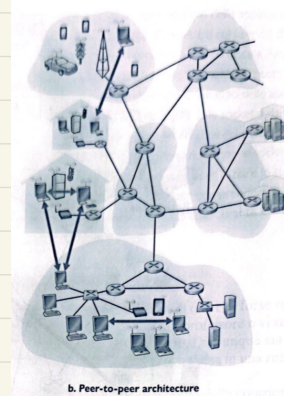
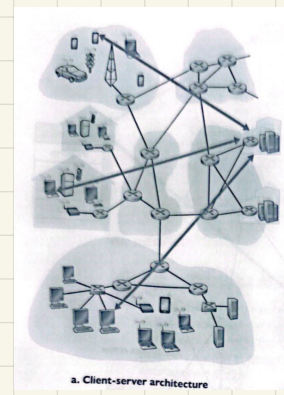
- i client non comunicano direttamente tra loro

- il server dispone di un indirizzo di rete fisso e noto, detto IP, a cui il client può sempre rifarsi per contattarlo

Spesso un singolo host che segue un server non è in grado di rispondere a tutte le richieste dei suoi client, per tale ragione si usano spesso i data Centre che, ospitando più host, creano un potente server virtuale

Nell'architettura P2P

- L'infrastruttura di server in data center è minima o del tutto assente
- Si sfrutta invece la comunicazione diretta tra coppie di host chiamate peer (ossia pari) collegate in modo intermittente (*Peer* = computer fissi e portatili)



Uno dei punti di forza di questa architettura è la sua intrinseca **scalabilità**, a ragione del fatto che ogni peer aggiunge capacità di servizio al sistema rispondendo a richieste di altri peer, risultando in un'architettura economicamente stabile perché:

- non richiedono per i server una significativa infrastruttura
- nè una disponibilità di banda elevata

2.1.2 - Processi comunicanti

Un altro aspetto da conoscere prima di costruire un'applicazione di rete e come comunicano tra loro i programmi in esecuzione su diversi host, programmi che nel gergo dei SO vengono detti **processi comunicanti**, dove per processo si intende un programma in esecuzione su un sistema

Quando i processi su due SO diversi desiderano comunicare tra di loro, lo fanno scambiandosi messaggi attraverso la rete:

- il processo mittente crea messaggi e li invia attraverso la rete
- il processo destinatario li riceve e, quando previsto, invia messaggi di risposta

Comprendiamo quindi che le applicazioni di rete sono costituite da una coppia di processi che si scambiano messaggi su una rete, coppia in cui etichettiamo:

- un processo come **client**, che è quello che avvia la comunicazione e
- l'altro come **server**, che è quello che attende di essere contattato per iniziare la sessione

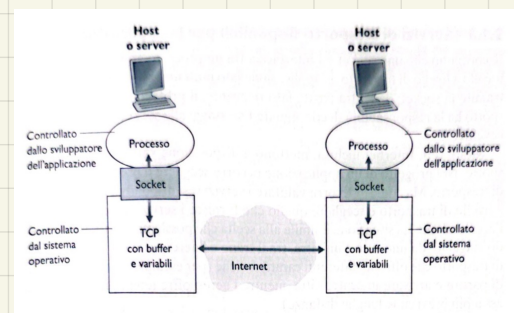
NOTA - ruoli nelle architetture

Client server
browser - client
web server - server

P2P
A seconda della situazione un peer può essere sia client che server

Per quanto riguarda i messaggi scambiati, essi passano attraverso un'interfaccia software detta socket, nonché l'interfaccia tra il livello di applicazione e il livello di trasporto all'interno di un host

Si parla anche di **API** (applicazioni Programming Interface) tra l'applicazione e la rete, dato che la socket rappresenta l'interfaccia di programmazione con cui applicazione di rete vengono costruite



Infine, per identificare il processo ricevente è necessario specificare due informazioni:

1. l'indirizzo dell'host e
2. un identificatore del processo ricevente sul l'host di destinazione

(Analogia casa (processo) e porta (socket),)
 (una casa che vuole inviare un messaggio a un)
 (altro casa fa uscire il messaggio dalla propria)
 (porta il quale passerà attraverso la rete fino)
 (ad arrivare alla porta del casa di destinazione)
 (che attraverserà per arrivare al casa ricevente)

Per quanto riguarda l'informazione (1), gli host vengono identificati attraverso i loro **indirizzi IP**, un numero di 32 bit che identifica univocamente l'host

Per quanto riguarda l'informazione (2), (Questa informazione aggiuntiva serve perché oltre a conoscere)
 questo compito viene assolto dal (l'indirizzo dell'host cui è destinato il messaggio, il mittente deve)
numero di porta di destinazione (anche identificare il processo destinatario, più specificatamente la)
 (socket che deve ricevere il dato, a ragione del fatto che sull'host)
 (potrebbero essere esecuzione molte applicazioni di rete)

Alle applicazioni più note sono stati assegnati numeri di porta specifici come

- web server - 80
- server di posta che usa il protocollo SMTP - 25
- ecc...

2.1.3. - Servizi di trasporto disponibili per le applicazioni

Abbiamo detto che quando un processo vuole mandare un messaggio ad un altro processo questo passa attraverso la rete, abbiamo omesso però come effettivamente avviene il passaggio e questo avviene attraverso un protocollo a livello di trasporto. La scelta del protocollo ricade sullo sviluppatore e vien fatta in base ai servizi offerti dal protocollo in questione e sulle ricadute che questi avranno sull'applicazione

I servizi che possono essere offerti possono essere generalizzati in 4 categorie:

- **Trasferimento affidabile**
- **Throughput**
- **Temporizzazione**
- **Sicurezza**
- **Trasferimento affidabile**
 Di fronte alla possibilità che i pacchetti possono andare perduti o danneggiati questo servizio è la soluzione a tal problema

Se un protocollo lo fornisce, allora si dice che fornisce un **trasferimento dati affidabile** (reliable data transfer)

ES - posta elettronica
messaggistica istantanea
trasmissione di file

I protocolli che non lo forniscono potrebbero essere usati dagli **applicazioni che tollerano le perdite** (loss-tollerante applications)

ES - applicazioni multi-mediali
audio/video a uso personale

- **Throughput** (quantità di dati al secondo che può essere trasferita tra due sistemi periferici)

Di fronte alla possibilità che alcune sessioni possano condividere la banda sul percorso di rete e che queste possano essere istituite e rilasciate dinamicamente facendo fluttuare nel tempo il throughput disponibile, alcune applicazioni potrebbero volere un throughput disponibile garantito

ES - applicazione di telefonia
codifica voce a 32 kbps

Distinguiamo:

- le applicazioni che hanno requisiti di throughput, vengono dette **applicazioni sensibili alla banda** (bandwidth-sensitive applications)
- le applicazioni che NON hanno requisiti, vengono dette **applicazioni elastiche** (elastic applications)

- **Temporizzazione**

Di fronte alla necessità che ogni bit che il mittente invia sulla socket venga ricevuto dalla socket di destinazione in un determinato intervallo di tempo, un protocollo a livello di trasporto potrebbe fornire garanzie di temporizzazione (timing)

Il timing interessa:

- telefonia Internet
- ambienti virtuali
- teleconferenze
- giochi multiutente

- **Sicurezza**

Di fronte alla possibilità che il messaggio possa essere catturato, un protocollo libero di trasporto può fornire a un'applicazione uno o più servizi di sicurezza come:

- la cifratura
- l'integrità di dati
- l'autenticazione

Applicazione	Tolleranza alla perdita di dati	Throughput	Sensibilità al fattore tempo
Trasferimento file/download	No	Variabile	No
E-mail	No	Variabile	No
Web	No	Variabile (pochi kbps)	No
Telefonia su Internet/ video conferenza	Sì	Audio: da pochi kbps a 1 Mbps Video: da 10 kbps a 5 Mbps	Sì: centinaia di ms
Streaming audio/video memorizzati	Sì	Come sopra	Sì: pochi secondi
Giochi interattivi	Sì	Fino a 10 kbps	Sì: centinaia di ms
Messaggistica istantanea	No	Variabile	Sì e no

2.1.4. - Servizi di trasporto offerti da Internet

Dopo aver esaminato i vari servizi offerti dai protocolli di trasporto, andiamo a vedere gli effettivi protocolli di trasporto, che sono:

- **TCP** e
- **UDP**

- **TCP**

Prevede:

- un servizio orientato alla connessione e
- il trasporto affidabile dei dati
 - Servizio orientato alla connessione (connection-oriented service)

Si intende che il client e il server si scambiano informazioni di controllo a livello di trasporto prima che i messaggi a livello di applicazione comincino a fluire. Questa procedura viene detta **handshaking** e serve per mettere in allerta client e server preparandoli alla partenza dei pacchetti

Dopo l'handshaking si dice che esiste una connessione TCP tra le socket dei due processi e tale è *full-duplex*, ovvero i processi possono scambiarsi contemporaneamente messaggi sulla connessione, connessione che verrà chiusa dall'applicazione quando terminerà di inviare messaggi

- Servizio di trasferimento affidabile (reliable data transfert service)
Come già anticipato, i processi comunicanti possono contare su TCP per trasportare i dati:
 - senza errori e
 - nel giusto ordine

- **UDP**

Prevede un servizio:

- NON orientato alla connessione (connectionless) e
- inaffidabile

UDP:

- NON svolge l'handshaking
- NON garantisce la consegna effettiva né in ordine dei pacchetti
- NON include un meccanismo di controllo della congestione pertanto esso può spingere i dati a livello sottostante (di rete) a qualsiasi velocità

NOTA - Focus sulla sicurezza

Né TCP né UDP forniscono forme di cifratura, alta ragione la comunità di Internet sviluppato un elemento aggiuntivo per TCP chiamato **Transport Layer Security (TLS)**

NOTA - Servizi non forniti dai protocolli di trasporto Internet

Nella descrizione dei servizi forniti da TCP e UDP mancavano cenni in merito a:

- garanzie di throuput e
- temporizzazione

Nonostante ciò, Internet può offrire un servizio soddisfacente le applicazioni sensibili ai ritardi, senza rispettare quei due criteri

Applicazione	Protocollo a livello di applicazione	Protocollo di trasporto sottostante
Posta elettronica	SMTP [RFC 5321]	TCP
Accesso a terminali remoti	Telnet [RFC 854]	TCP
Web	HTTP 1.1 (RFC 7230)	TCP
Trasferimento file	FTP [RFC 959]	TCP
Multimedia streaming	HTTP (per esempio: YouTube), DASH	TCP
Telefonia Internet	SIP [RFC 3261], RTP [RFC 3550], o proprietario (per esempio: Skype)	UDP o TCP

2.1.5. - Protocolli a livello di applicazione

Sempre riprendendo la parte in cui abbiamo spiegato che processi di rete comunicano tra di loro inviando messaggi tra socket, in merito ai messaggi abbiamo omesso:

- come sono strutturati
- il significato dei loro campi e
- quando vengono inviati

Tutti dettagli e modalità di scambio di messaggi che vengono definiti dai **protocolli a livello di applicazione**, che nello specifico o definiscono:

- i tipi di messaggi scambiati
- la sintassi dei vari tipi di messaggio
- la semantica dei campi
- le regole per determinare quando e come un processo invia e risponde ai messaggi

(se di richiesta o risposta)

(quali sono i campo nel messaggio
e come vengono descritti)
(il significato delle
informazioni che contengono)

Alcuni protocolli a livello di applicazione vengono specificati nelle RFC e pertanto sono di dominio pubblico (HTTP, protocollo a livello di applicazione del web - RFC 7230) e di conseguenza se lo sviluppatore di un browser si attiene alle regole che vi sono esposte il suo browser sarà in grado di recuperare pagine web da qualsiasi server che segua quelle stesse regole. Altri protocolli a livello di applicazione invece sono privati (skype)

NOTA - Differenza tra applicazioni di rete e protocolli a livello di applicaizone

Un protocollo a livello di applicazione è solo una parte di un'applicazione di rete

ES - Web, applicazione client-server è costituita da:

- uno standard per i formati di documento (HTML)
- un browser (chrome, Internet explorer)
- un web server (Apache e Microsoft)
- un protocollo a livello di applicazione (HTTP)

2.1.6 - Applicazioni di rete trattate in questo libro

Ogni giorno vengono sviluppate nuove applicazioni per Internet, pertanto ci concentreremo su quelle più diffuse e importanti come:

- **il Web (HTTP)**
- **la posta elettronica**
- **il servizio di directory (DNS)**
- **lo streaming video e**
- **le applicazioni P2P**

2.2 - Web e HTTP

Riepilogo storico

- Fino agli anni 90 Internet era conosciuto e limitato solo nel contesto universitario o di ricerca
- Nei primi anni 90 arriva il World Wide Web (WWW) [Berners Lee 1989] che spopola grazie al fatto che opera su richiesta (on demand), si può avere ciò che si vuole, quando lo si vuole e al fatto che era facilmente accessibile e a basso costo

2.2.1. - Panoramica di HTTP

HTTP (Hyper Text Transfer Protocol) è il protocollo a livello di applicazione del web e costituisce il cuore del web

Questo protocollo è implementato in due programmi:

- | | | | |
|----|--------------|---|---|
| 1. | nel client e |] | in esecuzione su sistemi periferici diversi che |
| 2. | nel server |] | comunicano tra loro scambiandosi messaggi HTTP |

HTTP definisce:

- sia la struttura dei messaggi
- sia la modalità con cui il client e il server si scambiano i messaggi

Prima di procedere, un po' di terminologia:

- una **pagina web** (web page) è costituita da oggetti

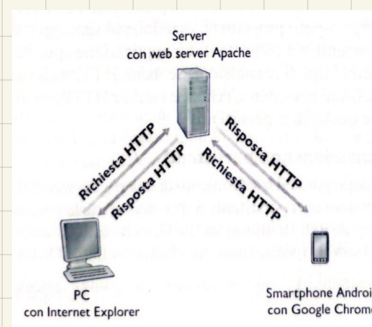
- un **oggetto** (object) è semplicemente un file (HTML, JPEG, Javascript ecc...) indicizzabile tramite un URL
- la maggioranza delle pagine web consiste di un **file HTML principale** (base HTML file) e diversi oggetti referenziali da esso, il file HTML referencia gli altri oggetti nella pagina tramite il loro URL
- ogni URL ha due componenti
 - il **nome dell'host del server** che ospita l'oggetto e
 - il **percorso dell'oggetto**

ES - <http://www.someSchool.edu/someDepartment/picture.gif>

- un **browser web** (chrome o Internet explorer) implementa il lato client di HTTP
- un **web server**, che implementa il lato server di HTTP, ospita oggetti web indirizzabili tramite URL

Fatto questo, descriviamo in maniera generale come avviene l'interazione client-server

- l'utente richiede una pagina web
- il browser invia messaggi di richiesta HTTP per gli oggetti nella pagina
- il server riceve le richieste
- il server risponde con messaggi di risposta HTTP contenente gli oggetti richiesti



Aggiungiamo che durante l'intera procedura:

- HTTP utilizza TCP a cui sia il client che il server accedono via le loro socket
- il server invia i dati richiesti senza memorizzare alcuna informazione di stato a proposito del client, per questo motivo HTTP è classificato come **protocollo senza memoria di stato** (stateless protocol)

2.2.2 - Connessioni persistenti e non persistenti

Di rado in molte applicazioni per Internet client e server comunicano per un lungo periodo di tempo, il client richiede e il server fornisce, ma la serie di richieste potrebbe essere effettuata:

- in sequenza, periodicamente a intervalli regolari o
- in maniera intermittente

Di fronte a questi due approcci di comunicazione, gli sviluppatori possano optare a due approcci diversi

1. inviare ciascuna coppia richiesta/risposta su una connessione TCP separata, in tal caso si dirà che l'applicazione usa **connessioni non persistenti**
2. inviare ciascuna coppia richiesta/risposta sulla stessa connessione TCP, in tal caso si dirà che l'applicazione usa **connessioni persistenti**

HTTP di default usa connessioni persistenti

1. connessioni persistenti

Supponiamo che la pagina consista di un file HTML principale e di 10 immagini JPEG e che tutti gli 11 oggetti risiedano sullo stesso server

Ipotizziamo che l'URL del file HTML principale sia http://www.someSchool.edu/someDepartment/home_index

Ecco che cosa avviene:

1. Il processo client HTTP inizializza una connessione TCP con il server www.someSchool.edu sulla porta 80, che è la porta di default per HTTP. Associate alla connessione TCP ci saranno una socket per il client e una per il server.
2. Il client HTTP, tramite la propria socket, invia al server un messaggio di richiesta HTTP che include il percorso / someDepartment / home. Index.
3. Il processo server HTTP riceve il messaggio di richiesta attraverso la propria socket associata alla connessione, recupera l'oggetto / someDepartment- / home. index dalla memoria (centrale o di massa), lo incapsula in un messaggio di risposta HTTP che viene inviato al client attraverso la socket.
4. Il processo server HTTP comunica a TCP di chiudere la connessione. Questo, però, non termina la connessione finché non sia certo che il client abbia ricevuto integro il messaggio di risposta.
5. Il client HTTP riceve il messaggio di risposta. La connessione TCP termina. Il messaggio indica che l'oggetto incapsulato è un file HTML. Il client estrae il file dal messaggio di risposta, esamina il file HTML e trova i riferimenti ai 10 oggetti JPEG.
6. Vengono quindi ripetuti i primi quattro passi per ciascuno degli oggetti JPEG referenziati.

Essendo la connessione non persistente ogni connessione TCP, che trasportava soltanto un messaggio di richiesta e risposta, è stata chiusa dopo l'invio dell'oggetto da parte del server, per un totale di 11 connessioni

Tuttavia, in merito alle 10 connessioni per ottenere i file JPEG, essi potevano essere ottenuti via in serie o in parallelo, infatti alcuni browser possono essere configurati per controllare il grado di parallelismo abbreviando i tempi medi di risposta

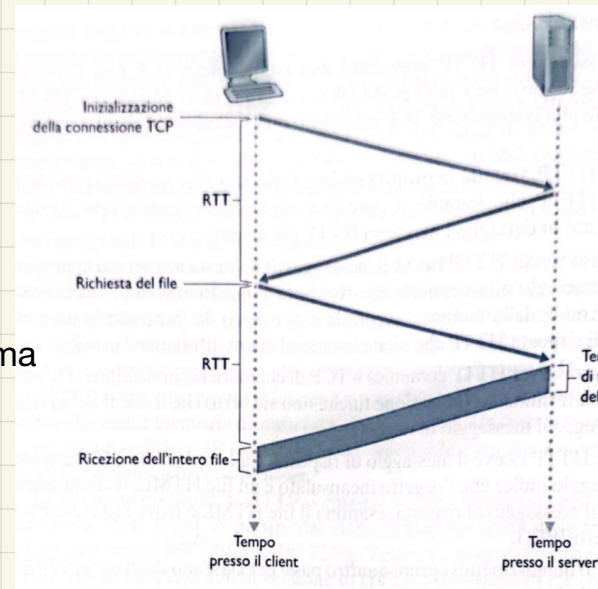
ES - se il grado di parallelismo è uguale a 1, allora le 10 connessioni saranno stabilite in serie

Parlando di tempistica, definendo il **Round Trip Time (RTT)**, il tempo impiegato da un piccolo pacchetto per viaggiare dal client al server per poi tornare al client, facciamo un calcolo approssimativo per stimare l'intervallo di tempo che intercorre tra le richieste di un file HTML da parte del client e il momento in cui l'intero file viene ricevuto dal client

RTT include ritardi:

- di propagazione
- di accodamento nei router e
- nei commutatore intermedi nonché di lavorazione del pacchetto

- per prima cosa il browser web inizializza una connessione TCP con il web-server comportando un **handshake a tre vie** (Thread-way handshake) dove:
 - il client invia un piccolo segmento TCP al server
 - quest'ultimo manda una conferma per mezzo di un piccolo segmento TCP
 - infine il client dà anch'esso una conferma di ritorno (acknowledgement) al server con un messaggio di richiesta HTTP
- In risposta al messaggio di richiesta il server inoltra il file HTML sulla connessione TCP



Tenendo conto che:

- le prime due parti dell'handshake a tre vie richiedono un RTT
- e così anche il messaggio di conferma/richiesta
- + il tempo di trasmissione da parte del server del file HTML

Il tempo totale è due RTT + tempo di trasmissione

Tuttavia si evidenziano alcuni limiti:

1. Per ogni oggetto richiesto occorre stabilire e mantenere una nuova connessione
2. per ciascuna di queste connessioni si devono allocare buffer e mantenere variabili TCP sia nel client sia nel server, con un conseguente peso sul web server che può dover servire contemporaneamente più richieste provenienti da altri client
3. Il tempo totale di consegna diventa di 2 RTT + il tempo di trasmissione

2. connessioni persistenti

Nelle connessioni persistenti:

- il server lascia la connessione TCP aperta dopo l'invio di una risposta, per cui altre richieste e risposte successive possono fare uso della stessa connessione su cui possono essere mandati più di un singolo oggetto e

- le richieste di oggetti possono essere effettuate una di seguito all'altra senza aspettare le risposte delle richieste pendenti (pipelining)

L'unico momento in cui la connessione viene chiusa è quando essa rimane inattiva per un dato lasso di tempo

HTTP di default impiega connessioni persistenti con pipelining

2.2.3 - Formato dei messaggi HTTP

Adesso andiamo a vedere velocemente i formati dei messaggi HTTP:

- di richiesta e
- di risposta
- **messaggio di richiesta HTTP**

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Messaggio scritto in testo ASCII così da essere leggibile dall'utente

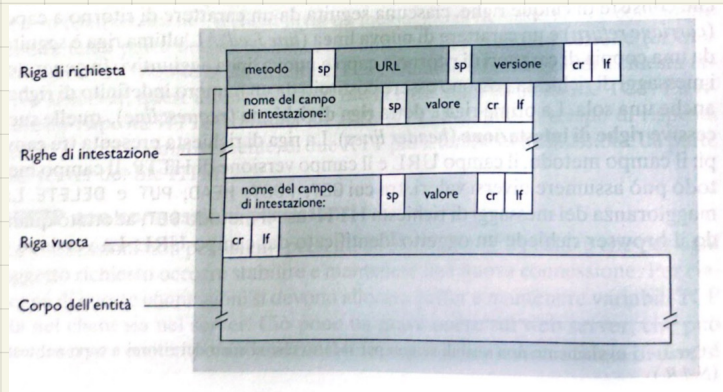
5 righe (anche se possono essere indefinite, anche 1)

Ciascuna seguita da:

- un carattere di ritorno a capo (carriste return) e
- un carattere di nuova linea (line feed)

- prima riga è detta **linea di richiesta** (request line) e presenta tre campi:
 1. **il campo metodo** (GET, POST, HEAD, PUT e DELETE)
 2. **il campo URL**
 3. **il campo versione HTTP**
- le successive **righe di intestazione** (header lines) e di queste:
 - nella riga "host" è specificato l'host su cui risiede l'oggetto
 - nella riga "Connection: close" il browser sta cominciando al server che non si deve occupare di connessioni persistenti, ma vuole chiudere la connessione dopo aver inviato l'oggetto richiesto
 - nella riga "User-agent" è specificato il tipo di browser che sta effettuando la richiesta al server
 - nella riga "Accept-language" viene specificato in che lingua l'oggetto sarebbe desiderato (in caso contrario il server lo manderebbe nella lingua di default)

Formato generale di un messaggio di richiesta



Dopo le linee di intestazione si trova un "corpo" (entity body) che:

- è vuoto nel caso del metodo GET
- viene riempito nel caso del metodo POST (quando l'utente riempie un form)

ES - voci da trovare a un motore di ricerca

In merito ai metodi:

- GET viene usato nei form HTML nel cui corpo sono inseriti i dati immessi nell'URL richiesto

ES - GET, due campi, scimmie e banane, allora l'URL ha la struttura

<http://www1.somesite.com/?tm=1&subid=1691908010.0364980000&searchbox=0&domainname=0>

- HEAD simile a GET, quando il server riceve la richiesta con HEAD risponde con un messaggio HTTP, ma trasmette gli oggetti richiesti (usato per verificare la correttezza del codice prodotto)
- PUT consente agli utenti di inviare un oggetto a un percorso specifico (directory) su uno specifico web e viene usato dalle applicazioni che richiedono di inviare oggetti ai web server
- DELETE consente la cancellazione di un oggetto su un server

ES - usato assieme agli strumenti di pubblicazione sul web

• Messaggio di risposta HTTP

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

Tre sezioni:

1. **riga di stato** (status line)
2. **6 righe di intestazione** (header lines) e
3. il **corpo** (entity body) [oggetto richiesto]

1. la **riga di stato** ha 3 campi
 - 1.1 **versione del protocollo**
 - 1.2 **codice di stato**
 - 1.3 **corrispettivo messaggio di stato**
(OK - tutto va bene, il server ha trovato l'oggetto e lo sta inviando)

```

HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)

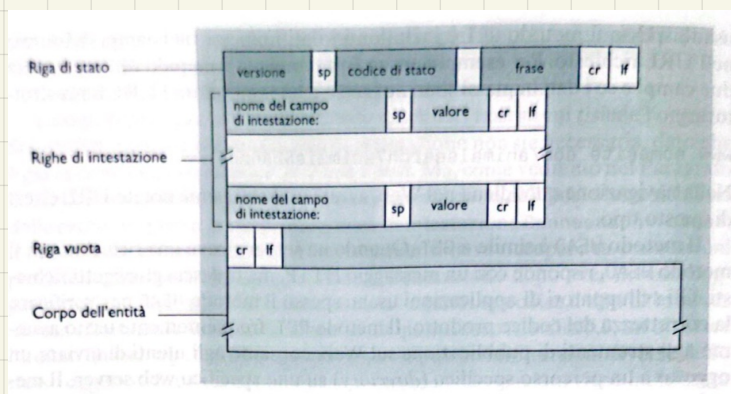
```

2. delle **righe di intestazione** distinguiamo:

- 2.1 nella riga “connection: close” viene comunicato al client che il server ha intenzione di chiudere la connessione TCP dopo l’invio del messaggio
- 2.2 nella riga “Date” viene indicata l’ora è la data di creazione e invio della risposta HTTP (nello specifico quando l’oggetto è stato recuperato, inserito e mandato nel messaggio di risposta)
- 2.3 nella riga “Server” viene indicato il server in cui la richiesta è stata ricevuta
- 2.4 nella riga “Last-modified” è indicato l’istante e la data in cui l’oggetto è stato creato o modificato per l’ultima volta
- 2.5 nella riga “Content-length” è indicato il numero di byte dell’oggetto inviato
- 2.6 nella riga “Content-type” è indicato in che forma e linguaggio si trova l’oggetto

Importante per la gestione dell’oggetto nella cache

Formato generale di un messaggio di risposta



Andiamo a vedere nello specifico i codici di stato e le loro espressioni che indicano il risultato della ricerca

Tra i più comuni troviamo:

- 200 OK - la ricerca ha avuto successo e l’informazione sta venendo inviata
- 301 Moved permanently - l’oggetto richiesto è stato trasferito in moto permanente, il nuovo URL è specificato nell’intestazione “Location” del messaggio di risposta, il cliente recupererà automaticamente il nuovo URL
- 400 Bad request - si tratta di un codice di errore generico che indica che la richiesta non è stata compresa dal server
- 404 Not found - il documento richiesto non esiste sul server
- 505 HTTP Version Not Supported - il server non dispone della versione di protocollo HTTP richiesta

2.2.4 - Interazione utente-server: i cookie

Ripasso

Aggiungiamo che durante l'intera procedura:

- HTTP utilizza TCP a cui sia il client che il server accedono via le loro socket
- il server invia i dati richiesti senza memorizzare alcuna informazione di stato a proposito del client, per questo motivo HTTP è classificato come **protocollo senza memoria di stato** (stateless protocol)

Nonostante abbiamo affermato che i server HTTP sono privi di stato alcuni spesso auspicano ad autenticare gli utenti e per fare ciò i server adottano i **cookie**, che permettono di tenere traccia degli utenti

La tecnologia dei cookie presenta 4 componenti:

1. una riga di intestazione nel messaggio di risposta HTTP
2. una riga di intestazione nel messaggio di richiesta HTTP
3. un file mantenuto sul sistema dall'utente e gestito dal browser e
4. un database sul sito

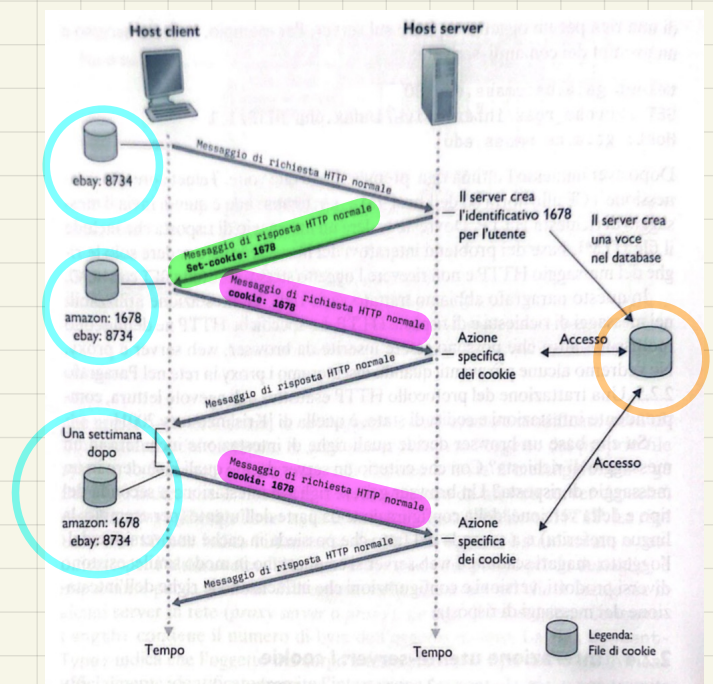
Il concetto di base è che al primo accesso ad un sito questo crea un **identificativo unico a voce nel proprio database indicizzato dal numero identificativo Set-cookie: xxx**

intestazione che quando il browser vede aggiunge una riga al **file dei cookie** che gestisce, riga che contiene:

- il nome dell'host del server e
- il numero identificativo nell'intestazione "Set-cookie"

Durante la navigazione in quello stesso sito, ad ogni richiesta di una pagina web il browser consulta il **file cookie dell'utente**, ne estrae il **numero identificativo** per il sito e pone nella richiesta HTTP una **riga di intestazione del cookie che include tale numero**
Cookie: xxx

così da monitorare l'attività dell'utente nel sito, anche senza sapere il suo nome saprà quali pagine ha visitato, in che ordine e quando



Monitorando l'attività il sito poi può anche arrivare a consigliare all'utente degli oggetti sulla base delle pagine che ha visitato e ancor peggio se l'utente si registrasse al sito al numero identificativo andrebbero associate più informazioni

ES - Amazon

cartello che tiene traccia degli oggetti acquisiti o presi in attesa di acquisto "one-click shopping", dopo aver immesso informazioni come il nome o numero carta di credito queste non vengono ulteriormente richieste

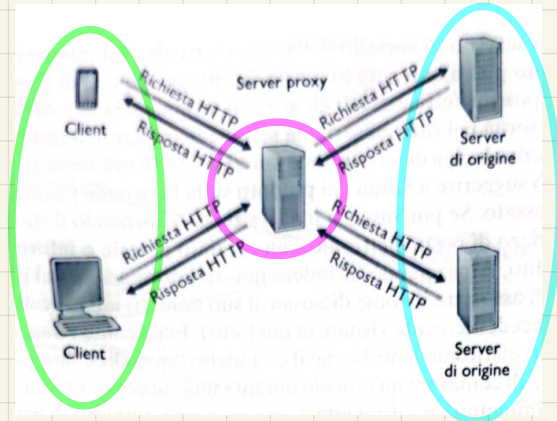
2.2.5 - Web caching

Una **web cache o proxy server** è un'entità di rete che soddisfa richieste HTTP al posto del web server effettivo

Esso ha una propria memoria su disco (una cache) in cui conserva copie di oggetti recentemente richiesti

Se il proxy viene configurato in maniera tale che il traffico passi attraverso ad esso:

- quando il **browser** richiede un oggetto la richiesta viene inviata al Proxy server
- il Proxy server controlla se è dell'oggetto richiesto c'è una copia memorizzata localmente
 - se dell'oggetto c'è una copia allora essa viene mandata al browser
 - se NON c'è:
 - il proxy invia una richiesta di quell'oggetto al **server di origine** e se la fa arrivare
 - arrivata, il proxy ne fa una copia nella sua cache e ne invia una al browser



Proxy si comporta:

- sia da client
- che da server

Così facendo:

- riduce sostanzialmente il traffico i tempi di risposta alle richieste dei client
- riduce il traffico sul collegamento di accesso a internet, così da non aumentare la banda e i costi

Tuttavia potrebbe subentrare il problema che la copia dell'oggetto che risiede nella cache potrebbe essere scaduta, ovvero l'originale da cui era stata fatta una copia potrebbe essere stato modificato rendendo la copia obsoleta

Per questo motivo HTTP è stato fornito di un meccanismo che permette alla cache di verificare se i suoi oggetti sono aggiornati e questo è chiamato **GET condizionale** (conditional GET)

Un messaggio di richiesta HTTP viene detto messaggio di GET condizionale se:

1. usa il metodo GET
2. include una riga di intestazione "If-modified-since:"

Funzionamento generale

- Quando un oggetto viene richiesto, di questo non c'è la copia e quindi viene mandata dal server di origine con conseguente copia, dell'oggetto copiato viene anche salvata la data di ultima modifica (Last-modified)

- Quando arriva il momento di controllare se l'oggetto originale è stato modificato nel messaggio di richiesta GET condizionale nella riga di intestazione "If-modified-since" viene messa la data dell'ultima modifica precedentemente salvata e viene richiesto che l'oggetto venga rinviato solo se è stato modificato rispetto alla data specificata
- Se non è stato modificato il web server invia un messaggio di risposta con "codice di stato" 304 (Not modified) e corpo vuoto altrimenti lo manda

2.2.6 - HTTP/2

Prima nuova versione di HTTP da HTTP/1

HTTP/1 faceva uso di connessioni TCP persistenti (un'unica connessione TCP utilizzata) pertanto non solo il numero di socket sul server era limitato, ma si scoprì il problema dell'**Head of Line (HOL) blocking** (blocco in testa di coda), ovvero se su una singola connessione TCP venisse mandato un file di ingenti dimensioni questo rallenterebbe o peggio bloccherebbe l'invio di altri file anche se di piccole dimensioni

Per questo motivo i browser HTTP/1 furono dotati della possibilità di aprire connessioni TCP parallele in maniera tale che gli altri file piccoli potessero essere inviati

In aggiunta il controllo della congestione era un incentivo per utilizzare connessioni parallele poiché assegnava una quota uguale di larghezza di banda alle connessioni parallele

Tuttavia nell'aprire più pagine web il browser può "imbrogliare" e prendersi una parte maggiore della larghezza di banda del collegamento

Per questo motivo fu creato HTTP/2, per:

- in primis per eliminare/ridurre il numero di connessioni TCP parallele per il trasporto di una singola pagina web (così da ridurre numero di socket da aprire e mantenere sui server e far funzionare correttamente la congestione)
- ma anche per
- ridurre latenza percepita attivando il multiplexing di richiesta
 - ridurre numero di risposte su una singola connessione TCP
 - fornire supporto per la priorità delle richieste e il server Push e
 - fornire una comprensione efficiente dei campi l'intestazione HTTP

La soluzione di HTTP/2 al problema dell'**HOL** consiste nel suddividere ogni messaggio in piccoli frame e alternare i messaggi di richiesta e risposta sulla stessa connessione TCP per poi ricostruire il tutto a destinazione, questo processo prende il nome di **framing**

ES - dato un oggetto X di 1000 frame e più oggetti piccoli Y di 2 frame, quando un frame di X viene mandato uno di Y viene mandato a sua volta, quindi al secondo frame di X un oggetto Y sarà stato mandato interamente

Il processo di framing viene seguito dal sottolivello di framing del protocollo HTTP/2, in cui una risposta HTTP viene elaborata e scomposta in frame nel server, mentre nel client vengono man mano ricostruiti nei messaggi di risposta originali per poi essere elaborati come da procedura. Oltre a fare ciò il sottolivello framing esegue anche una codifica binaria dei frame perché così sono più efficienti da analizzare e sono meno soggetti ad errori

Altri meccanismi di HTTP/2 sono:

- **Priorità dei messaggi**
consente agli sviluppatori di indicare per quali messaggi si desidera la priorità in termini di consegna, il grado di priorità viene indicato assegnando alle risposte che richiede un peso di tra 1 e 256 a ciascun messaggio (256 = priorità assoluta)
- **Server pushing**
è la capacità di un server di inviare più risposte per una singola richiesta del cliente, cioè oltre alla risposta alla richiesta originale, il server può effettuare un invio “push” al client di oggetti aggiuntivi, senza che il client debba richiedere ognuno di essi, eliminando così latenza aggiuntiva dovuta all’attesa per le richieste

ES - pagina web
HTML indica gli oggetti necessari per il rendering e questi vengono mandati prima che vengano richiesti

2.2.7 - HTTP/3

QUIC è un nuovo protocollo di “trasporto” implementare un livello di applicazione sul protocollo UDP

Offre diverse funzionalità per HTTP come:

- il multiplexing dei messaggi
- il controllo di flusso per ogni stream e
- la creazione di connessioni a bassa latenza

HTTP/3 è un nuovo protocollo HTTP progettato per funzionare su QUIC, anche se non è stato completamente standardizzato

2.3 - Posta elettronica in Internet

Una delle più importanti “*Killer application*” di internet

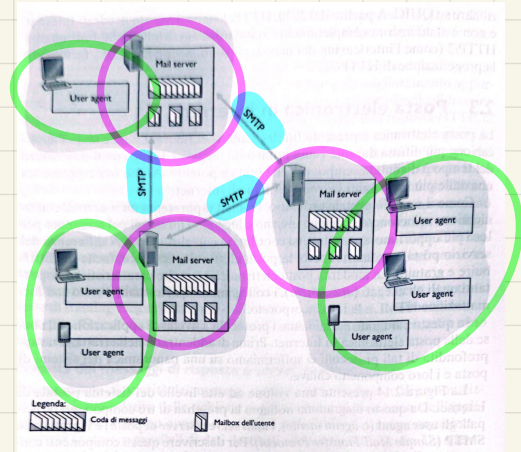
- veloce] altre caratteristiche molto importanti:
- facile da distribuire e] • gli allegati (attachments) • testo con formattazione HTML e
- gratuita] • i collegamenti ipertestuali • le foto incorporate

3 componenti principali:

1. **gli user agent** (agenti utente)
2. **i mail server** (server di posta)
3. **e il protocollo SMTP**
(Simple Mail Transfer Protocol)

di cui:

1. gli **user agent** sono le entità responsabili della composizione, modifica, lettura e dell'invio del messaggio scritto da un utente al mail server, dove verrà posto nella coda di messaggi in uscita o del recupero di un messaggio inviatoci
2. i **mail server** costituiscono la parte centrale dell'infrastruttura del servizio di posta elettronica ed in essi è collocata una **mail-box** (casella postale) che ogni utente possiede e che gestisce e contenimento i messaggi a lui inviati (è qui che l'user agent recupera le email)



user agent mittente → mail server mittente → mail server destinatario → user agent destinatario

Per accedere ai messaggi della propria mail-box si deve essere autenticati via nome e password

Il mail server del mittente deve eventualmente gestire i problemi del mail server del destinatario, se non potesse ricevere messaggi quello da inviare viene trattenuto in una **coda di messaggi** (message queue) per poi essere trasferito in un secondo momento (tentativi ogni 30 minuti, dopo alcuni giorni di fallimenti rimuove il messaggio e notifica la mancata consegna al mittente via un messaggio di posta elettronica)

3. **SMTP** è il protocollo a livello di applicazione per la posta elettronica e:
 - trasferisce i messaggi dal mail server del mittente a quello del destinatario
 - fa uso del trasferimento dati affidabile
 - essendo a livello applicazione presenta
 - un lato client (esecuzione lato mittente, quando invia posta) e
 - un lato server (esecuzione lato destinatario, quando riceve posta)

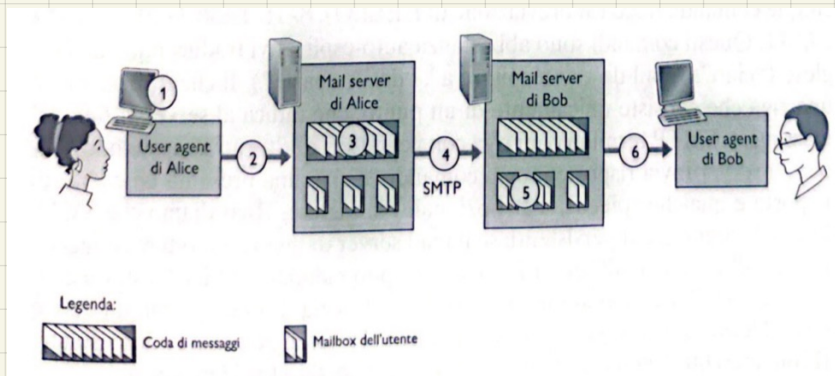
Problema collocazione mail server sull'host locale
Se si facesse così gli host comunicerebbero direttamente tra loro, ma entrambi per ricevere dovrebbero rimanere sempre accessi, per questo si preferisce che ci sia un user agent che accede ad un mail server condiviso da altri utenti, ma almeno sempre accesso

2.3.1 - SMTP

Tratta il corpo di tutti i messaggi di posta come semplice ASCII a 7 bit, sensata in passato, ma ora restrittiva in quanto richiedere i dati multimediali binari vengono codificati in ASCII prima di essere inviati per poi essere decodificati in binario dopo il trasporto

Il funzionamento consiste:

- che il mail server del mittente, nel momento in cui deve inviare un messaggio al mail server del destinatario, apre una connessione TCP verso un server SMTP in esecuzione sul mail server del destinatario
- dopo un handshaking SMTP il client SMTP invia il messaggio sulla connessione TCP
- presso il mail server del destinatario il server SMTP riceve il messaggio che viene posizionato nella casella postale del destinatario



NOTA - NO mail server intermedi

La connessione viene stabilita solo e soltanto tra i mail server degli interlocutori, anche se lontanissimi

ES - trascrizione messaggi scambiati tra:

- un client (C - crepes.fr) e
- un server SMTP (S - hamburger.edu)

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Ti piace il ketchup?
C: Che cosa ne pensi dei cetrioli?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Comandi

NOTA - Il messaggio non viene mandato direttamente

Se lo fosse, l'utente mittente usa SMTP per spingere i messaggi nel mail server che a sua volta userà SMTP per spingere il messaggio al mail server del destinatario. Il motivo è perché in caso di errore nella consegna, l'utente mittente non saprebbe cosa fare in caso di errore nella consegna, ma essendoci il mail server come intermedio prima della consegna effettiva, esso ha un riferimento perché il messaggio è depositato e può riprovare a mandarlo.

2.3.2 - Formati dei messaggi di posta

From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Alla ricerca del significato della vita.

linea vuoto

"Corpo"

2.3.3 - Protocolli di accesso alla posta

Per prendere il messaggio depositato dal mail server ci sono 2 modi:

1. se la posta elettronica è basata sul web o un app per smartphone, verrà usato HTTP
2. nel caso in cui i messaggi vengano recuperati da dei server si utilizza il protocollo **IMAP** (Internet Mail Access Protocol)

NOTA - Differenza HTTP e SMTP

- HTTP: client pull
 - SMTP: client push
- Entrambi hanno comandi/interazioni di risposta e codici di stato in ASCII
- HTTP incapsula ogni oggetto nel suo messaggio di risposta
 - SMTP manda più messaggi in messaggi a più parti

2.4 - DNS: il servizio di directory di Internet

Gli host in Internet possono essere identificati in due modi:

1. via **hostname** anche se questi:
 - non danno molte informazioni sulla collocazione all'interno di Internet
 - non sono costituiti da un numero variabile di caratteri alfa numerici, pertanto sono difficilmente elaborazioni dai server
2. via **indirizzi IP**, di cui
 - sono di 4 byte
 - hanno una rigida struttura gerarchica
 - hanno una forma del tipo
121.7.106.83
in cui ogni punto separa uno dei byte espressi con un numero decimale compreso tra 0 e 255 e leggendoli da sinistra a destra si ottengono informazioni più specifiche sulla collocazione dell'host in Internet

ES - www.eurecom.fr
fr = Francia, suffisso del paese

2.4.1 - Servizi forniti da DNS

Quindi esistono due modi per identificare gli host:

1. via il nome, preferito dalla persone
2. via gli indirizzi IP preferito dai router

Al fine di conciliare i due approcci esiste il **DNS** (Domain Name System), un servizio in grado di tradurre gli hostname nei loro indirizzi IP che:

- è un database distribuito implementato in una gerarchia di **DNS server** e
- un protocollo a livello di applicazione che] usato da
consente agli host di interrogare il database] HTTP e SMTP

questa funzionalità prende il nome di "query", interrogazione e viene fatta dal DNS su richiesta del client

Oltre alla traduzione altri servizi sono:

- **Host aliasing**

Nel caso in cui hostname avesse dei sinonimi, anche più facili rispetto a quello originale, quest'ultimo viene categorizzata come **hostname canonico** e la scelta se utilizzare o uno o i suoi sinonimi ricade dall'utente che ha richiesto l'indirizzo IP

- **Mail server aliasing**

Seppur di base gli indirizzi di posta elettronica è consigliato siano facili da ricordare, non sempre è così e in maniera analoga al funzionamento dell'host aliasing lo stesso può essere fatto per gli indirizzi IP degli host

- **Distribuzione del carico di rete (load distribution)**

Serve per distribuire il carico tra server replicati, server replicati che nonostante ognuno abbia un host diverso e un indirizzo IP differente, a tutti gli indirizzi IP, contenuti nel server DNS, è associato un hostname canonico. Quando i client fanno la query DNS per un nome associato a un insieme di indirizzi, il server risponde con l'intero insieme l'indirizzi, ma ne varia l'ordinamento ad ogni risposta

2.4.2 - Panoramica del funzionamento di DNS

Superficialmente, quando un client vuole tradurre un hostname in un indirizzo IP, l'host invoca il lato client del DNS specificando l'hostname da tradurre (`gethostbyname()`). A quel punto il DNS invia un messaggio di richiesta, query, sul rete, a cui riceve una risposta contenente la corrispondenza richiesta che poi viene passata all'host

Al primo sguardo si potrebbe consigliare un unico server DNS tutto centralizzato, tuttavia, di fronte ad un vasto e sempre crescente numero di host, si potrebbero riscontrare i seguenti problemi:

- **un solo point of failure** (punto di vulnerabilità)

Se il DNS server si guastasse ne soffrirebbe l'intero Internet

- **volume di traffico**

Un singolo DNS server dovrebbe gestire tutte le richieste, che siano HTTP o di posta elettronica

- **database centralizzato distante**

Un singolo DNS server non può essere vicino a tutti i client

- **manutenzione**

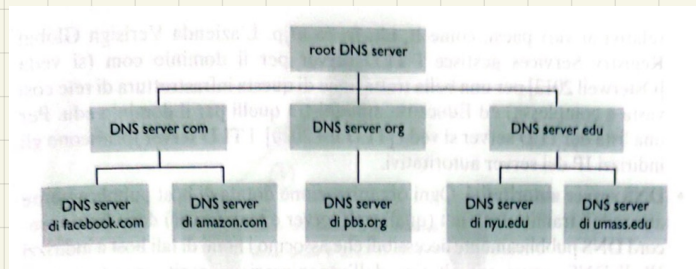
Un singolo DNS server dovrebbe contenere record relativi a tutti gli host di Internet, per fare ciò non solo dovrebbe essere vasto, ma anche costantemente aggiornato per tenere conto di ogni nuovo host

Comprendiamo quindi che il DNS non è scalabile

Come già detto il DNS è un database distribuito implementato in una gerarchia di DNS server distribuita nel mondo, ma di cui ognuno di essi non ha tutte le corrispondenze per tutti gli host di internet

La gerarchia è strutturata in 3 classi di DNS server:

1. i **root server**
2. i **top-level domain (TDL) server** e
3. i **server autoritativi**

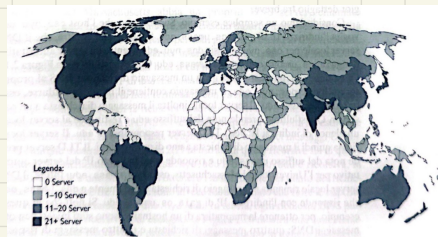


Supponendo di volere l'indirizzo IP di www.amazon.com:

1. il cliente dapprima contatta uno dei **root server** per trovare i **TDL server** per il dominio "com" (detto "dominio di primo livello" o "top-level")
2. a questo punto contatta il **TDL server** per avere l'indirizzo IP dei **server autoritativi** per amazon.com
3. infine contatta uno dei **server autoritativi** che gli restituisce l'indirizzo IP dell'host name

Dei server aggiungiamo che:

1. i **root server**
ne esistono più di 1000 e sono copie di 13 differenti root server gestiti da 12 diverse organizzazioni coordinate attraverso la *Internet Assigned Numbers Authority (IANA)*



2. i **top-level domain (TDL) server**
si occupano dei d'omicidio di primo livello come:
 - com
 - org
 - net
 - edu
 - gov e
 - domini relativi ai vari paesi (it, uk, fr...)

3. i **server autoritativi**

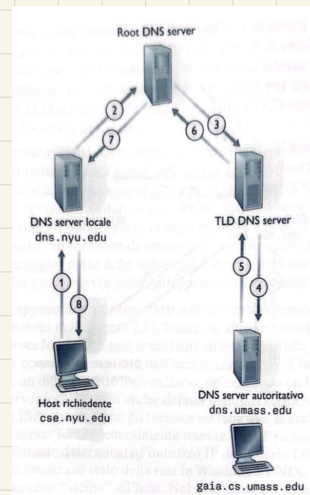
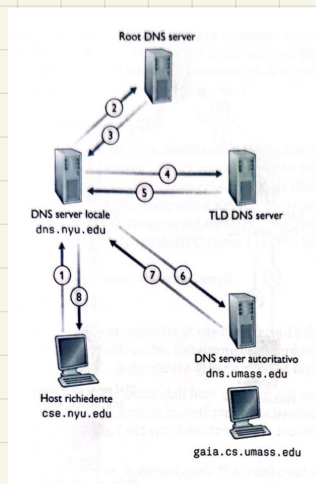
Sono i server che contengono tutti i record DNS che permettono di associare gli indirizzi IP agli host Name di ogni organizzazione data di host pubblicamente accessibili tramite Internet

Oltre a questi server ne esiste un altro che non appartiene per forza alla gerarchia, ma comunque è importante:

- **DNS server locale (default name server)**

il nome già descrive la sua funzione, l'unica cosa da aggiungere è che le query richieste possono essere:

- **query ricorsive** (query al DNS server locale) o
- **query iterative** (restanti query)



Un'altra funzionalità è la **DNS caching**, tralasciata, ma comunque fondamentale per DNS che la sfrutta in modo estensivo:

- per migliorare le prestazioni di ritardo e
- per ridurre il numero di messaggi DNS che rimbalzano su internet

2.4.3 - Record e messaggi DNS

I server che implementano il database distribuito di DNS memorizzano i cosiddetti **record di risorsa** (RR, resource record), tra cui quelli che forniscono le corrispondenze tra nomi e indirizzi

Un record di risorsa contiene i seguenti campi (Name, Value, Type, TTL)

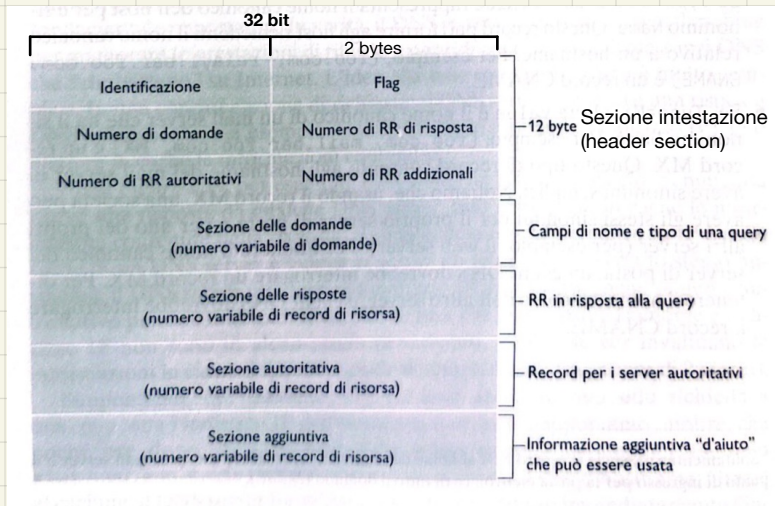
dove:

- TTL è il Time To Live, ossia il tempo di vita di un record e determina quando una risorsa vada rimossa dalla cache
- Name e Value dipendono da Type

Se:

- Type = A
 - Name = hostname] fornisce la corrispondenza tra
 - Value = indirizzo IP (dell'hostname)] l'hostname e l'indirizzo IP
- Type = NS
 - Name = dominio] viene usato per instradare le richieste DNS successive
 - Value = hostname del server autoritativo per il dominio] alla prima nella concatenazione delle query
- Type = CNAME
 - Name = alias per l'hostname canonico] viene usato per dare agli host richiedenti
 - Value = hostname canonico] il nome canonico relativo a un host name
- Type = MX
 - Name = alias per l'hostname canonico] viene usato per consentire agli hostname
 - Value = hostname canonico di un mail server] dei mail server di avere sinonimi semplici

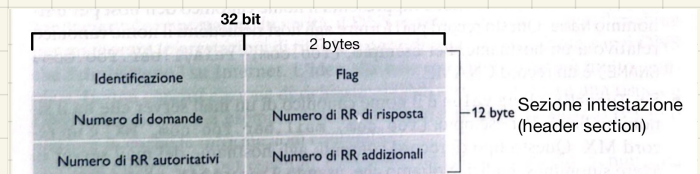
Per quanto riguarda i messaggi di risposta e anche le query, entrambi hanno lo stesso formato che è strutturato come mostrato



- sezione di intestazione (12 byte), che ha 6 campi

1. identificazione

identifica la richiesta e viene copiato nei messaggi di risposta a una richiesta, consentendo al cliente di far corrispondere le risposte ricevute con le query inviate

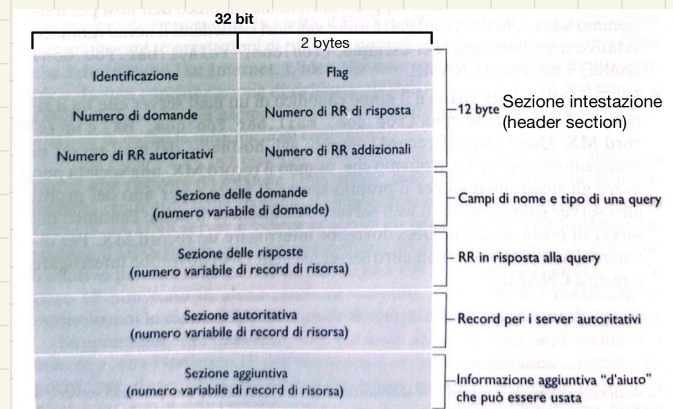


2. flag (sequenza di bit)

- Il primo di questi indica se il messaggio è
 - di richiesta, query - 0
 - (quando il DNS è autoritativo)
 - risposta, reply - 1
- un ulteriore bit è la **richiesta di ricorsione** (recursion-desired flag) viene impostato quando un client desidera che il DNS server effettui ricorsione quando non dispone del record

3. 4. 5. e 6. Che cominciano con "numero di" indicano il numero di occorrenze delle quattro sezioni tipo dati che seguono l'intestazione

- la sezione delle domande contiene informazioni sulle richieste che stanno per essere effettuate Include anche:
 - un campo nome con il nome che sta per essere richiesto e
 - un campo tipo che indica il tipo di domanda sul nome
- la sezione delle risposte Contiene i record di risorsa relativi al nome originariamente richiesto
- la sezione autoritativa Contiene i record di altri server autoritativi
- la sezione aggiuntiva Racchiude altri record utili



2.4.3.1 - Inserimento di record nel database DNS

Supponiamo che abbiamo appena fondato e avviato una nuova società chiamata “network utopia”

La prima cosa da fare per inserire il suo record nel database DNS è quello di registrare il nome di dominio “networkutopia.com” presso un ente di registrazione detto “**registrar**”, che è un’azienda che verifica l’unicità del nome di dominio

Quando il dominio sarà stato registrato, sempre al registrar si dovranno fornire anche i nomi e gli indirizzi IP dei nostri DNS server autoritativi primario e secondario. Se questi fossero

- dns1.networkutopia.com
- dns2.networkutopia.com
- 212.2.212.1 e
- 212.212.212.2.

Per ciascuno di questi due server autoritativi l’ente si accerterà dell’inserimento di un record di tipo NS di tipo a nei TLD server relativi al suffisso com., più nello specifico per il server autoritativo primario di networkutopia.com il registrar inserirebbe nel sistema DNS i due seguenti record di risorsa:

(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)

Oltre a questo dovremmo accertarci che:

- il record di risorse di tipo A per
il nostro web server e] vengono immessi nei
- che il record di risorse di tipo
mx per il nostro server di posta] nostri DNS server

Una volta completati questi passi sarà possibile visitare il nostro sito web e spedire posta elettronica ai dipendenti della nostra società

2.4.3.2 - Sicurezza di DNS

I DNS server possono essere attaccati via:

- **Attacchi DDOS**
attraverso richieste iterative
- **Redirect attacks**
intercettando le query (si risolve con HTTP)
- **DNS poisoning**
si chiede qualcosa al DNS locale che aspetta risposta, ma se ricevo una risposta da un DNS falso, la salva in cache infettando molti altri host

Si può anche usare un DNS per effettuare un attacco DDOS in cui l’IP sorgente della query è della macchina da attaccare. Per evitare questo non si accettano richieste da utenti non autorizzati

Infine

Ogni dominio ha un DNS primario e uno secondario, con il cui il primario si sincronizza. Fatta una query, se il primario non risponde allora si contatta il secondario e tra i due vi è una **zona di trasferimento** in cui si può inserire un host che può buttare giù il primario dal secondario. Quando un host proverà a contattare il primario, essendo quest'ultimo spento, allora contattare il secondario che, essendo alterato, porterà in siti differenti da quelli richiesti dall'host